

12주차 · 11/19(목)

HTML5 DOM 활용

DOM 구조 이해 및 요소 제어

한신대학교 AI·SW계열 · 웹프로그래밍
학습사이트 hs-web.dreamitbiz.com

학습 목표

- ① DOM이 무엇인지 설명하고 HTML 문서와의 관계를 말할 수 있다.
- ② querySelector로 요소를 찾고 내용·속성·클래스를 바꿀 수 있다.
- ③ textContent와 innerHTML의 차이와 보안 위험을 설명할 수 있다.
- ④ 요소를 만들어 붙이고 지울 수 있다.
- ⑤ querySelector가 null을 돌려주는 원인을 진단할 수 있다.

12주차

핵심 키워드

DOM

querySelector

textContent

innerHTML

XSS

classList

dataset

createElement

학습사이트의 본문에서 이 낱말들을 찾아 읽으면 이 시간의 뼈대가 잡힙니다

12주차 · 1

DOM — 브라우저가 만든 문서의 지도

브라우저는 HTML 파일을 읽어 객체들의 나무(tree)로 바꿔 메모리에 올립니다.

DOM — 브라우저가 만든 문서의 지도

- 1 이것이 DOM(Document Object Model) 입니다.
- 2 JavaScript는 이 나무를 타고 다니며 원하는 가지를 찾아 바꿉니다.

HTML이 나무가 되는 모습

```
1  <!-- HTML 파일 -->
2  <body>
3    <h1 id="title">제목</h1>
4    <ul class="list">
5      <li>하나</li>
6      <li>둘</li>
7    </ul>
8  </body>
9
10 /* 브라우저가 만든 DOM 나무
11 document
12 └─ html
13     └─ body
14         └─ h1#title
15             └─ "제목"      ← 텍스트 노드
16         └─ ul.list
17             └─ li → "하나"
18             └─ li → "둘"   */
```

꼭 구분해야 할 것

- 1 HTML 파일**
은 서버에 있는 글자 뭉치입니다 — 바뀌지 않습니다.
- 2 DOM**
은 브라우저 메모리 안의 객체 나무입니다 — JavaScript가 바꾸는 것은 이쪽입니다.
- 3 그래서 JS로 화면을 바꿔도 새로고침하면 원래대로 돌아옵니다. 원본 파일은 그대로니까요.**

12주차 · 2

요소 찾기

querySelector — CSS 선택자를 그대로 씁니다

```
1 document.querySelector('#title');    // id — 하나
2 document.querySelector('.list');     // class — 첫 번째 하나
3 document.querySelector('li');        // 태그 — 첫 번째 하나
4 document.querySelector('.list > li'); // 복잡한 선택자도 그대로
5
6 document.querySelectorAll('li');     // 전부 (NodeList)
7
8 // 못 찾으면 null을 돌려줍니다 — 이게 오류의 시작점입니다
9 const el = document.querySelector('#없는아이디');
10 console.log(el);                    // null
11 el.textContent = '안녕';            // ❌ TypeError: Cannot set properties of null
```

요소 찾기

- 1 getElementById · getElementsByClassName 같은 옛 방법도 있지만, `querySelector` 하나만 알면 됩니다.
- 2 CSS에서 쓰던 선택자를 그대로 쓸 수 있어 외울 것이 없습니다.

① 선택자 오타 ② script가 너무 일찍 실행됨

```
1 <!-- ❌ head에 있는 script는 body보다 먼저 실행됩니다 -->
2 <head>
3   <script>
4     document.querySelector('#btn'); // null — 아직 btn이 안 만들어졌다
5   </script>
6 </head>
7 <body>
8   <button id="btn">버튼</button>
9 </body>
10
11 <!-- ✅ 해결 1: script를 </body> 바로 앞에 -->
12 <!-- ✅ 해결 2: defer 속성 — <script src="app.js" defer></script> -->
13 <!-- ✅ 해결 3: DOM이 다 만들어진 뒤 실행 -->
14 <script>
15   document.addEventListener('DOMContentLoaded', () => {
16     document.querySelector('#btn'); // 이제 있습니다
17   });
18 </script>
```

querySelectorAll은 배열이 아닙니다

```
1  const items = document.querySelectorAll('li');
2
3  items.forEach(li => console.log(li.textContent)); // ✅ forEach는 됩니다
4  items.map(li => li.textContent);                 // ❌ map은 없습니다
5
6  const arr = [...items]; // 배열로 바꾸면 전부 쓸 수 있습니다
7  arr.map(li => li.textContent); // ✅
```

12주차 · 3

내용과 모양 바꾸기

글자 바꾸기

```
1  const title = document.querySelector('#title');
2
3  title.textContent = '바뀐 제목';      // 글자만 — 안전 ✅
4  title.innerHTML = '<em>기울임</em>';  // HTML로 해석 — 조심 ⚠
5
6  // 읽기도 됩니다
7  console.log(title.textContent);
```

innerHTML을 조심해야 하는 이유 (XSS)

- 1 innerHTML 은 넣은 문자열을 HTML로 해석합니다. 그래서 사용자가 쓴 글을 그대로 넣으면 그 안의 `<script>` 나 `onerror` 가 실제로 실행됩니다.
- 2 이것을 XSS(Cross-Site Scripting) 공격이라 하며, 남의 로그인 정보를 훔치는 데 쓰입니다.
- 3 원칙: 사용자가 입력한 것은 반드시 `textContent` 로 넣습니다. innerHTML 은 내가 직접 쓴 문자열에만 씁니다.

입력과 값

```
1  const input = document.querySelector('#name');
2
3  input.value           // 입력칸의 현재 값 (항상 문자열!)
4  input.value = '기본값';
5  Number(input.value)   // 계산하려면 변환 필수
6
7  const check = document.querySelector('#agree');
8  check.checked         // 체크박스는 checked (true/false)
```

스타일 — 클래스로 바꿉니다

```
1  const box = document.querySelector('.box');
2
3  // ❌ 이렇게 직접 만지지 마세요 — 스타일이 JS 안에 흩어집니다
4  box.style.background = 'red';
5  box.style.padding = '20px';
6
7  // ✅ 스타일은 CSS에 두고, JS는 클래스만 바꿉니다
8  box.classList.add('is-active');
9  box.classList.remove('is-active');
10 box.classList.toggle('is-active');           // 있으면 빼고 없으면 넣기
11 box.classList.toggle('is-active', true);      // 강제로 켜기
12 box.classList.contains('is-active');          // 있는지 확인
```

속성 다루기

```
1  img.src = 'new.jpg';
2  link.href = 'https://example.com';
3
4  el.setAttribute('data-id', '42');
5  el.getAttribute('data-id');      // '42'
6  el.dataset.id;                  // '42' ← data-* 는 dataset으로 더 편하게
7  el.removeAttribute('disabled');
8
9  // 보이기/숨기기 — hidden 속성이 가장 간단합니다
10 panel.hidden = true;
11 panel.hidden = false;
12 /* △ CSS에 [hidden] { display: none !important } 를 함께 두세요.
13     다른 display 규칙이 hidden을 이겨 버리는 일이 실제로 자주 있습니다. */
```

12주차 · 4

요소 만들고 붙이기

createElement + append

```
1  const li = document.createElement('li'); // 만들고
2  li.textContent = '새 항목';              // 채우고
3  li.className = 'item';
4  document.querySelector('ul').append(li); // 붙인다
5
6  // 여러 개
7  ['하나', '둘', '셋'].forEach(text => {
8    const li = document.createElement('li');
9    li.textContent = text;
10   ul.append(li);
11 });
12
13 // 지우기
14 li.remove(); // 자기 자신 제거
15 ul.textContent = ''; // 자식 전부 비우기 (가장 간단한 방법)
```

요소 만들고 붙이기

메서드	어디에
<code>append(el)</code>	자식의 맨 뒤
<code>prepend(el)</code>	자식의 맨 앞
<code>before(el) / after(el)</code>	자기 앞 / 뒤 (형제로)
<code>replaceWith(el)</code>	자기 자리를 대체
<code>remove()</code>	자기 자신 제거

12주차

실습

이 시간의 실습 3개 — 학습사이트의 코드 실습장에서 직접 치고 실행합니다

실습 1 · 기본

JS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

“버튼을 누르면 h1의 글자가 바뀌게 하세요. HTML은 JavaScript로 만들어도 되고, `document.body.innerHTML`로 먼저 넣어도 됩니다.

힌트

`querySelector`로 찾고 `addEventListener`로 걸고 `textContent`로 바꿉니다.

실습 1 모범답안

```
1 document.body.innerHTML = '<h1 id="title">처음 제목</h1><button id="btn">바꾸기</button>';
2
3 document.querySelector('#btn').addEventListener('click', () => {
4     document.querySelector('#title').textContent = '바뀐 제목! 🎉';
5 });
```

실습 2 · 기본

JS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

“목록을 JavaScript로 만들어 화면에 붙이세요. 배열 ["HTML","CSS","JavaScript"]를 돌면서 li를 만들어 ul에 넣습니다. 사용자 입력이 아니어도 `textContent`를 쓰세요.

힌트

`createElement`로 만들고, `textContent`로 채우고, `append`로 붙입니다.

실습 2 모범답안

```
1 document.body.innerHTML = '<h2>배우는 것</h2><ul id="list"></ul>';
2
3 const items = ['HTML', 'CSS', 'JavaScript'];
4 const ul = document.querySelector('#list');
5
6 items.forEach((text) => {
7     const li = document.createElement('li');
8     li.textContent = text;
9     ul.append(li);
10 });
```

실습 3 · 응용

JS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

“버튼을 누르면 상자에 클래스가 붙었다 떨어지게 하세요. 스타일은 CSS에 두고 JavaScript는 클래스만 토글해야 합니다. 지금 상태를 화면 글자로도 표시하세요.

힌트

`classList.toggle()`은 붙었는지 여부를 `true/false`로 돌려줍니다. 그 값을 그대로 쓰면 편합니다.

실습 3 모범답안

```
1 document.body.innerHTML = `  
2   <style>  
3     .box { padding: 24px; border: 2px solid #cbd5e1; border-radius: 12px; transition: .2s; }  
4     .box.is-on { background: #4338ca; color: #fff; border-color: #4338ca; }  
5   </style>  
6   <div class="box" id="box">상자</div>  
7   <button id="btn">켜기 / 끄기</button>  
8   <p id="state"></p>  
9 `;  
10  
11 const box = document.querySelector('#box');  
12 const state = document.querySelector('#state');  
13  
14 document.querySelector('#btn').addEventListener('click', () => {  
15   const on = box.classList.toggle('is-on');  
16   state.textContent = on ? '켜짐' : '꺼짐';  
17 });
```

12주차 마무리

12주차 마무리

HTML5 DOM 활용 — 이 시간에 다룬 것을 학습사이트에서 다시 확인합니다

✓ 학습사이트 주차 페이지의 절마다 「이해했어요 · 실습했어요 · 어려워요」 표시

✓ 퀴즈 10 「퀴즈 10 — HTML5 DOM 활용」 응시 (성적 미반영 · 학습 확인용)

✓ 실습 3개를 실습장에서 「확인」 까지 — 「실습했어요」 체크

✓ 막히면 「질문있어요」 또는 패들릿에 — 30분 넘게 혼자 막혀 있지 않기

다음 시간

13주차 · 이벤트 처리 기초 및 응용